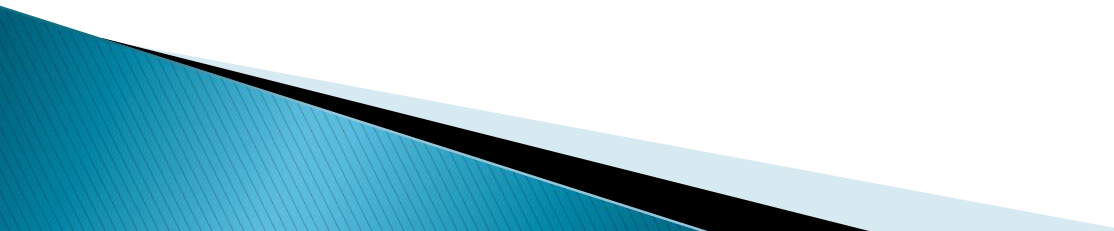


Жадные алгоритмы

Жадные алгоритмы

- ▶ Частичное решение расширяется на каждом шаге до тех пор, пока не будет получено полное решение
 - ▶ Идея стратегии – на каждом шаге получать максимальный (минимальный) «выигрыш», без нарушения ограничений задачи
 - ▶ Последовательность локально оптимальных шагов приводит к оптимальному решению всей задачи
- 

Пример: задача о выборе заявок (интервальное планирование)

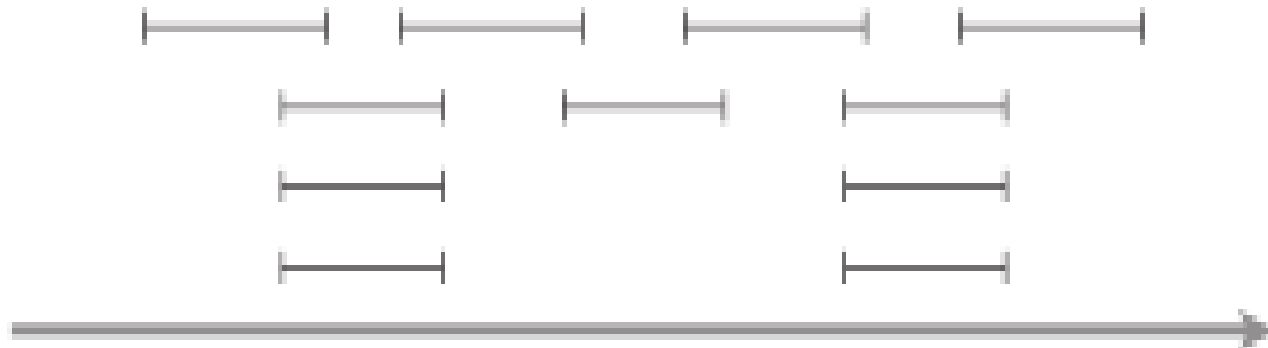
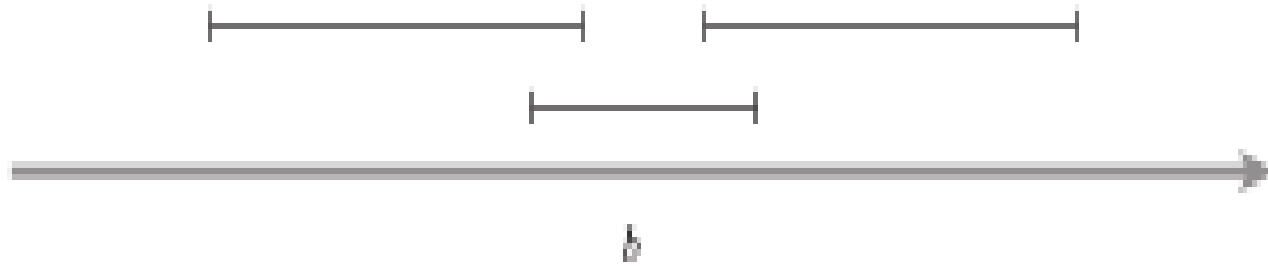
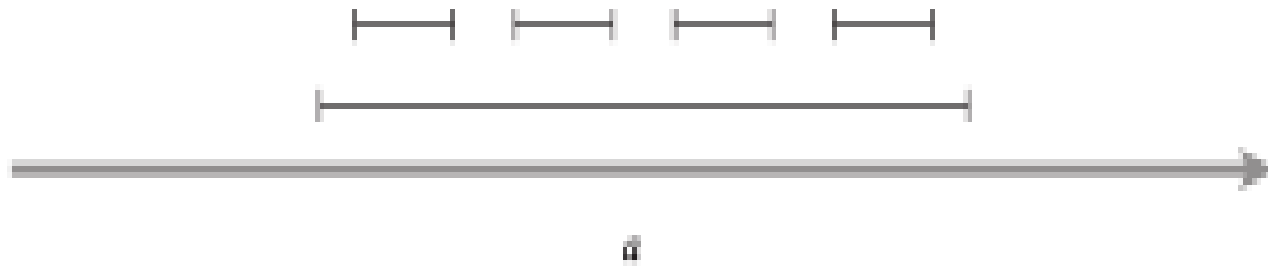
- ▶ Даны n заявок на проведения занятий в одной и той же аудитории. Два разных занятия не могут перекрываться по времени. В каждой заявке указаны начало и конец занятия (s_i, f_i для i -заявки). Разные заявки могут пересекаться, и тогда можно удовлетворить только одну из них. Конец одного занятия может совпадать с началом другого

Пример: задача о выборе заявок (интервальное планирование)

- ▶ Заявки с номерами i и j совместны, если интервалы $[s_i, f_i)$ $[s_j, f_j)$ не пересекаются ($f_i \leq s_j$ или $f_j \leq s_i$)
- ▶ Задача о выборе заявок состоит в том, чтобы набрать максимальное количество совместных друг с другом заявок

Правила

1. Выбирать доступную заявку с самым ранним начальным временем – заявку с минимальным значением $s(i)$
2. Заявку с наименьшим интервалом времени – разность $f(i) - s(i)$ оказывается наименьшей из всех возможных
3. Для каждой заявки подсчитывается количество других несовместимых заявок и принимается заявка с минимальным количеством несовместимых результатов. (интервал с наименьшим количеством «конфликтов»)



Жадное правило

- ▶ Жадное правило, которое приводит к оптимальному решению, строится на четвертой идее: первой принимается заявка, которая завершается первой (то есть заявка i с наименьшим значением $f(i)$). Идея вполне естественная: она гарантирует как можно более раннее освобождение ресурса и при этом удовлетворяет одну заявку. Тем самым обеспечивается максимизация времени, оставшегося для удовлетворения других заявок

Жадное правило

- ▶ Пример. 11 интервалов в порядке возрастания времени окончания: (1, 4), (3, 5), (0, 6), (5, 7), (3, 8), (5, 9), (6, 10), (8, 11), (8, 12), (2, 13), (12, 14)
- ▶ Ответ: (1, 4), (5, 7), (8, 11), (12, 14)

Предполагаем, что заявки упорядочены по времени окончания в порядке неубывания: $f_1 \leq f_2 \leq \dots \leq f_n$.

Сортировка, время $O(n \cdot \log n)$

процедура solve:

{Вход: массивы s , f – начала и конца заявок, упорядочены по возрастанию $f[i]$, $i=1..n$ }

{Выход: массив a – номера заявок, принятых к обслуживанию, $count$ – количество принятых заявок}

$j=1$, $a[1]=1$, $count=1$ {Первая заявка включается в решение}

для i от 2 до n :

если $s[i] \geq f[j]$:

$count++$

$a[count]=i$

$j=i$

Схема доказательства

(общая для всех задач, решаемых по данной стратегии)

- ▶ Шаг 1 – существует оптимальное решение, содержащее заявку с номером 1.
Действительно, если в каком-то оптимальном решении заявка с номером 1 не содержится, то можно заменить в нем заявку с самым ранним временем окончания на заявку с номером 1, что не повредит совместности заявок и не изменит их общего количества. Стало быть, можно искать оптимальное решение заявок, среди множеств, содержащих заявку номер 1. Существует оптимальное решение, начинающееся с жадного выбора

Схема доказательства

- ▶ Шаг 2. После того как стало ясно, что можно рассматривать подмножества, содержащие заявку с номером 1, выкинем все несовместимые заявки с заявкой с номером 1. Задача сводится к выбору оптимального выбора заявок из множества оставшихся заявок (совместимых с заявкой с номером 1). Другими словами, задача сведена к аналогичной, с меньшим числом заявок. Рассуждение по индукции